

AIM: To configure and demonstrate how a phishing-related Security event is detected and correlated.

OBJECTIVE:

01. To configure a SIEM Tool (Splunk) for detecting phishing-related Security events.
02. To analyze log data for Suspicious activities such as phishing emails.
03. To identify user identification & interactions with phishing links (clicks).
04. To detect failed login attempts resulting from phishing attacks.
05. To correlate multiple events to identify a complete phishing attack scenario.

PROCEDURE:

Step 01: Install and launch Splunk Enterprise and accessed it via the web interface.

Step 02: Once the program is installed, login to the application.

Step 03: Once the user logs in, They are directed to "Home page" in splunk.

Step 04: Before moving forward, create a sample phishing log file named "Phishing_logs.csv" containing fields such as:

- a. timestamp
- b. user
- c. IP address
- d. email details
- e. URL
- f. status.

Else, download a phishing email dataset from the internet.

Step 05: Once the file is created or downloaded, go back to Splunk.

Step 06: Upload the log file into Splunk using the "Add data" → "Upload" option stored in the "Main Index".

Step 07: Set the "Source Type" to "CSV". And the user can preview the data here.

Step 08: Once the Source Type is Set, Modify the Input Settings: There are 2 main sections:

- a. Host → where each event receives a "host" value. which can be set to
 - Constant value
 - Regular expression or path
 - Segment in path.
- b. Index → which has three options
 - history
 - Main
 - Summary.

Step 09: Review the settings. Then click on "next" to upload the document successfully.

Step 10: The user gets a notification saying the "File has been uploaded successfully".

Step 11: Click on "start Searching" to look for suspicious phishing attempts.

Step 12: To verify Data, Run:

index = main

in Search & Reporting. You will see all the logs present in the file.

Step 13: Detect phishing indicators using SPL (Splunk Processing Language).

index = main email_from! = "*@company.com"

This query finds email senders.

Step 14: Detect phishing URL clicks by using:

index = main status = clicked.

It shows the users who clicked the phishing link.

Step 15: Detect failed login after clicking

index = main Status = login_failed

This indicates possible theft attempt.

Step 16: Correlate phishing related events, using SPL correlation query:

index = main

| stats values (Status) as actions by user, src-ip, URL | search actions = "delivered" AND actions = "clicked" AND actions = "login_failed"

This Query:

- Groups logs by user
- checks if all phishing stages occurred.
- Detects Successful phishing attack chain.

Step 17: once done, Save it as either/or as:

- Dashboard
- Report
- create Alert.

Step 18: To create Dashbord, click

- Save → Dashbord Panel → New from report
as
- Select or Give it Title "phishing Detection"

c. click Save

once the dashboard is created, It will display the events detected.

Step 19: To create an "Alert", Use the same query. Click on "Save As → Alert".

a. Give the title "Phishing Attack Detected"

b. description (optional)

c. Permissions

- Private
- Public

d. Alert type

- Scheduled
- Real-time

e. Expiration.

f. Trigger conditions.

g. Trigger Alerts.

- Add to Triggered Alerts.
- Dashboard Studio Snapshot.
- Log event.
- Output results to lookup
- Output results to telemetry endpoint
- Run a script.

h. severity level when triggered.

OBSERVATION:

- a. The uploaded phishing log data was successfully indexed and searchable in Splunk.
- b. Queries identified suspicious external email senders and user interactions with phishing links.
- c. Multiple events such as email delivery, link clicks, and failed login attempts were observed for the same user.
- d. The correlation query successfully combined these events to identify a potential phishing attack.
- e. The dashboard visually represented phishing activities, and the alert triggered when the defined conditions were met.

RESULT:

- a. Phishing log data was successfully uploaded and indexed in Splunk.
- b. Suspicious email activity from external senders was detected.
- c. User interactions such as clicking phishing links were identified.
- d. Failed login attempts after link clicks were observed.

e. correlation of multiple events (delivery, click, login failed) was achieved.

f. A phishing attack scenario was successfully detected using SPL queries.

g. Alerts were generated automatically based on the correlation condition.

h. The experiment demonstrated effective phishing detection using Splunk SIEM.

AIM: To install, configure, and implement the Intrusion Detection and Prevention system using Snort.

OBJECTIVES:

- a. understand the primary uses of snort as a packet sniffer, packet logger, and full blown network intrusion prevention system.
- b. Successfully install snort and its requirements components, such as Winpcap, on window environment.
- c. Configure the snort.conf file to define protected network variables and set appropriate absolute paths for rules and logs.
- d. Develop and implement custom rules in the local.rules file to identify various network scanning attacks.
- e. Validate the NIDS by stimulating attacks using tools like Nmap or Zmap to ensure real-time alert generation.

PROCEDURE:

Step 1: Download the Snort installer from the official website and follow the Setup wizard, ensuring Snort, Dynamic modules, & Documentation are selected. Install Winpcap 4.1.1 as part of the process.

Step 2: Open the command prompt, navigate to "bin" directory.

```
cd c:/snort
```

↓

```
cd bin.
```

Step 3: To confirm the Version & Successful Installation; run:

```
snort -V
```

Step 4: To Download the rules Snapshot from the snort website, and extract it to "c:/snort"

Step 5: To find the IP address of the machine, run:

```
ipconfig
```


Step 6: To Edit "Snort.conf" file ,

Go to " c:/snort/etc " and open "snort.conf" file using Notepad.

Step 7: There are a total 9 steps to edit in Snort.conf file. ~~5~~ ~~6~~ ~~7~~

Step 8: Set the network variables in "Step 1" of Snort.conf file by typing the IP address found in Step 5.

Step 9: Set up the "external network Address as " \$HOME_NET".

Step 10: Set the path of the rules files as " c:/snort/rules" and preprocessing rules as " c:/snort/preproc_rules".

Step 11: Set the white-list & black-list path as " c:/snort/rules".

Step 12: configuring the decoder in step 2 of "snort.conf" by setting the path of log directory as " c:/snort/log".

Step 13: There are no changes in "Step 3" of "snort.conf" file.

Step 14: configure dynamic loaded libraries in "Step 4" of "snort.conf" by setting the path of:

a. dynamic preprocessor → "c:/snort/lib/snort-dynamicpre-processor."

b. base preprocessor engine → "c:/snort/lib/snort-dynamicengine/sf-engine.dll".

Step 15: configure preprocessors in "Step 5" of "snort.conf" by removing the "1" in "webroot no"

Step 16: Put "decompress_swf" & "decompress_pdf" in comments.

Step 17: Put "processor bo" in comments.

Step 18: Delete comment from preprocessor "sfportscan".

Step 19: set path to whitelist & blacklist.

create a new white list & blacklist.

Save these files in rules directory:

"c:/snort/rules".

Step 20: There is no change in Step 6 (configure output plugins) of "snort.conf" file.

Step 21: Customize rule set in "Step 7" of "snort.conf" file by replacing the forward slash "/" with "backslash" "\".

* Only applicable for windows OS *

Step 22: Customize preprocessor & decoder alerts in "Step 8" of "snort.conf" by replacing "/" with "\" (backward slash).

Step 23: There is no change in "Step 9" (Customize shared object snort rules) of "snort.conf" file.

Step 24: Open the command prompt & go to "c:\snort\bin;" & type "snort -W" to check the available interface.

Step 25: Execute the snort tool in command prompt by typing:

"snort -i 2 -c C:/snort/etc/snort.conf",

Step 26: After running snort in IDS mode, the next step is to write rules in "local.rules" file.

Step 27: Execute snort IDS mode by typing

"snort -i 1 -c C:/snort/etc/snort.conf -A console" in the command prompt & press enter.

Step 28: Perform network scanning attacks with nmap by typing "nmap -p 1-65535 -v 127.0.0.1" in the command prompt.

Step 29: The network scanning attacks can be performed using "znmmap tool"

COUNTERMEASURES:

- a. Always disable SNMP and SMB on hosts if not using it for a particular period of time.
- b. Block the SNMP ports (UDP ports 161 + 162) and SMB ports (TCP port 139 + 445) at the network perimeter.
- c. There's Technically a "U" that's a part of the solution: upgrade. Upgrading Systems (at least the ones you can) to SNMP Version 3 and SMB Version 2 can resolve many of the well-known SNMP & SMB security weaknesses.

OBSERVATION:

- a. Snort successfully initializes by parsing the configuration file & loading all active rules, including those in "local.rules".
- b. When the Nmap scan is initiated, the snort console immediately begins packet processing.

c. The console display real-time alerts for each detected scan attempt, showing the timestamp, rule-message (e.g. "SYN attack"), priority level, protocol type, and the source / destination IP addresses.

RESULT:

a. The Snort NIDS was successfully installed and verified on the windows platform.

b. The system was correctly configured to monitor the local network and log alerts to the designated directory.

c. custom rules effectively detected and provided alerts for simulated network scanning attacks, proving the system's efficacy in identifying potential instructions.

RG

AIM: Advance Enumeration Password cracking using hashcat.

OBJECTIVES:

- a. Understand the fundamental properties of password hashing, including its deterministic, one-way, and fixed-length nature.
- b. Identify the differences between various hash algorithms like MD5, NTLM, and SHA-1 regarding their security and processing speed.
- c. Learn to set up a controlled environment by creating custom wordlists and hash files in a linux-based system.
- d. Master the execution of different attack modes, specifically dictionary attacks, for known wordlists & mask attacks for pattern-based brute forcing.

PROCEDURE:

Step 1: Create a md5-hash.txt with the first hash.

echo '560b0b14200ecd8c02554281f6cf d28' > md5-hash.txt

Step 2: Create ntlm-hash.txt with the NTLM hash

```
echo 'bc8d5b7ee1db26e1974f0857e794383d' > ntlm-hash.txt
```

Verify using:

```
cat ntlm-hash.txt
```

Step 3: Create pin-sha1hash.txt with the SHA-1 hash

```
echo 'aee655773d856f b0 38536adcf6472fc7543463e' >
```

```
pin-sha1hash.txt
```

Verify using:

```
cat pin-sha1hash.txt
```

Step 4: Create a demo wordlist using:

```
nano demo_wordlist.txt
```

and include list of passwords to crack in file.

Step 5: Crack MD5 hashes using dictionary attack using:

```
hashcat -a 0 -m 0 md5-hash.txt demo_wordlist.txt  
--potfile-disable
```


Step 6: Crack NTLM hash using dictionary attack

```
hashcat -a 0 -m 1000 ntlm_hash.txt demo_wordlist.txt --potfile-disable.
```

Step 7: Crack SHA-1 PIN using brute force (all 4-digits combos)

```
hashcat -a 3 -m 100 pin_sha1hash.txt ?d?d?d?d
```

OBSERVATION:

- Hashcat Successfully matched hashes in the dictionary attack because the corresponding password were present in the wordlist.
- The mask attack ~~effectively~~ exhausted all 10,000 possible combination (0000 - 9999) to find the correct 4-digit PIN.
- The terminal output displayed a "cracked password" for all three attempts, providing the results in a "hash: plaintext" format.
- session details confirmed -the recovery rate (100 %), time elapsed (0 secs), and the specific hardware devices utilized for the cracking passwords

RESULT:

- a. The MD5 hash successfully cracked to: nfsuchennai@2026.
- b. The NTLM hash successfully cracked to: Nfsu#2025!
- c. The SHA-1 4-digit PIN hash successfully cracked via brute-force to: 2026.

RTJ

PRACTICAL - 04

01. **AIM:** To demonstrate stack and heap buffer overflow and analyze their effects using GDB.

02. **OBJECTIVES:**

- Understand stack and heap buffer overflow.
- Observe memory corruption due to oversized input.
- Use GDB to inspect memory & variables.
- compare behavior of Stack vs heap overflow.

03. **PROCEDURE:**

Step 1: created two C programs:

a. Stack Overflow using fixed-size buffer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main ()
```

```
{
```

```
    char buffer [8];
```

```
    int marker = 12345;
```

```
    printf ("marker before input: %d\n", marker);
```

```
    printf ("Enter some text: ");
```

```
    scanf ("%s", buffer);
```

```
    printf ("You entered: %s\n", buffer);
```

```
    printf ("Marker after input: %d\n", marker);
```

```
    return 0;
```

```
}
```

b. Heap Overflow using malloc ():

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
int main ()
```

```
{
```

```
    char *buf1 = (char *) malloc (8);
```

```
    char *buf2 = (char *) malloc (8);
```

```
    printf ("Addresses: \n");
```


OUTPUT:

Heap overflow:

a. Normal Case:

manosri@ManosriSara: /mnt/c/users/manos/downloads \$

gcc -g heap-overflow-demo.c -o heapdemo

manosri@ManosriSara: /mnt/c/users/manos/downloads \$

./heapdemo

Before Overflow: SAFE

Enter text: hello

After input:

buf1 = hello

buf2 = SAFE

b. Overflow Case:

manosri@ManosriSara: /mnt/c/users/manos/downloads \$

gcc -g heap-overflow-demo.c -o heapdemo

manosri@ManosriSara: /mnt/c/users/manos/downloads \$

./heapdemo

Before Overflow: SAFE

Enter text: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

After input:

buf1 = AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

buf2 =

double free or corruption (out)

Aborted (core dumped)


```

printf ("buf2 = %p\n", buf2);

strcpy (buf2, "SAFE");

printf ("InBefore Overflow: \n");
printf ("buf2 = %s\n", buf2);

printf ("Enter text: ");
scanf ("%s", buf1);

printf ("InAfter input: \n");
printf ("buf1 = %s\n", buf1);
printf ("buf2 = %s\n", buf2);

free (buf1);
free (buf2);

return 0;
}

```

Step 2: Compile with debug Symbols using:

```
gcc -g -fno-stack-protector Stack_overflow_demo.c -o demo
```

```
gcc -g heap_overflow.c -o heapdemo
```

Step 3: Executed programs with

- Small input → normal behavior.
- Large input → Overflow condition.

Step 4: Used GDB to inspect execution using,

```
gdb ./demo
```

```
gdb ./heapdemo
```

Commands used:

- break main
- run
- next
- printer & buffer
- print & marker
- print buf1
- print buf2
- continue.

4. OBSERVATION:

- a. Stack overflow caused stack corruption & program termination with "Stack Smashing detected".
- b. Heap overflow caused heap corruption & termination with "double free or corruption".
- c. Small inputs worked normally.
- d. Large inputs exceeded buffer size & corrupted memory.
- e. GDB help observe variable addresses and memory behavior.

5. RESULT:

The experiment successfully demonstrated both stack and heap buffer overflow. It was observed that exceeding buffer limits leads to memory corruption & abnormal program termination. GDB proved useful in analyzing memory & understanding overflow behaviour.

16/7

PRACTICAL - 05

01. AIM: To Perform WLAN Penetration testing using kismet and analyze nearby wireless networks for security vulnerabilities.

02. OBJECTIVES:

- a. To understand wireless network monitoring concepts.
- b. To install and configure kismet tool.
- c. To capture & analyze WLAN Traffic.
- d. To identify available wireless networks & their properties.
- e. To evaluate network security (encryption types, hidden SSIDs, etc)

03. PROCEDURE:

Step 1: Install kismet.

- a. open terminal (Linux recommended)
- b. Run :

Sudo apt update

Sudo apt install kismet.

Step 2: Start kismet using:

- a. Launch kismet using:

Sudo kismet

- b. kismet will open a web interface.

Step 3: configure Wireless Interface.

- a. Select your Wireless adapter (e.g. wlan0)
- b. Ensure it supports monitor mode.

Step 4: Begin Scanning

- a. Start scanning for nearby wireless networks.

- b. kismet will automatically detect:

- SSID
- BSSID
- channel
- Signal strength

Step 5: Analyze Networks

(a) observe different types of networks:

- open networks
- WPA/WPA2 Secured networks.
- Hidden SSIDs

(b) Identify:

- weak or unsecured networks.
- Signal Strength Variations.
- Duplicate or Suspicious access points.

Step 6: capture Packets

(a) Kismet captures packets automatically.

(b) logs are stored in:

/var/log/kismet

Step 7: Stop the Scan

(a) Press `ctrl+c` to stop Kismet

(b) Review captured logs & reports.

4. OBSERVATIONS:

(a) Multiple wireless networks were detected in the vicinity.

(b) Networks were identified with different encryption types: Open (no encryption), WPA/WPA2 Secured.

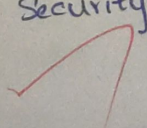
(c) Signal strength varied based on distance

(d) Hidden SSIDs were detected through probe responses.

(e) Some networks showed strong signals but weak security configurations.

5. RESULT:

The WLAN pentesting using Kismet was successfully performed. Various wireless networks were identified & analyzed based on their security configurations. The practical demonstrated how unsecured or weakly secured networks can be detected, highlighting the importance of proper wireless security measures.



Roby

PRACTICAL - 06

01. AIM: To implement data encryption and decryption using the SHA-512 Hashing algorithm and the RSA Algorithm, and to understand their roles in ensuring data security.

02. OBJECTIVES:

- To understand the concept of data encryption and decryption.
- To study the working of SHA-512 hashing algorithm for data integrity.
- To implement ~~RSA~~ algorithm for secure encryption and decryption.
- To compare hashing and encryption techniques.
- To ensure secure transmission of data using cryptographic methods.

03. PROCEDURE:

Step 1: Environment Setup

- Install Python & required libraries (hashlib, rsa or cryptography)
- Open any IDE (VS code / Py charm / cmd)

Step 2: Implement SHA-512 Hashing using

```
import hashlib

#input message
message = input("Enter message to hash: ")

#convert message to bytes
message_bytes = message.encode()

# Generate SHA-512 hash
hash_object = hashlib.sha512(message_bytes)
hash_value = hash_object.hexdigest()

# Display hash
print("SHA-512 Hash: ", hash_value)
```


Step 3: Implement RSA Encryption & Decryption

- Generate public & private keys.
- Encrypt plaintext using public key.
- Decrypt ciphertext using private key.

code:

```
import rsa
```

```
# Generate public & private keys.
```

```
public-key, private-key = rsa.newkeys(512)
```

```
# Input message
```

```
message = input("Enter message to encrypt:")
```

```
# Encrypt message
```

```
encrypted_message = rsa.encrypt(message.encode(), public-key)
```

```
# Decrypt message
```

```
decrypt_message = rsa.decrypt(encrypted_message, private-key).decode()
```

```
# Display results
```

```
print("Encrypted Message:", encrypted_message)
```

```
print("Decrypt Message:", decrypt_message)
```

Step 4: Verification

(a) check whether:

- original message == Decrypted message
- Hash value is generated correctly.

(b) Modify input slightly & observe change in hash value.

4. OBSERVATIONS:

- The SHA-512 algorithm generates a fixed length (512) bit hash value regardless of input size.
- The hash value is irreversible, meaning original data cannot be retrieved from it.
- RSA encryption successfully converts plaintext into unreadable

Ciphertext.

- d. Decryption using the private key retrieves the original message accurately.
- e. Even a small change in input produces a completely different hash value.

5. RESULT:

The implementation of SHA-512 & RSA Algorithm was Successfully Carried out.

- (a) Data was Securely encrypted & decrypted using RSA.
- (b) Data integrity was ensured using SHA-512 hashing.
- (c) The experiment demonstrated the importance of combining encryption (confidentiality) & hashing (integrity) in modern cryptographic systems.

Rev

PRACTICAL - 07

01. To Investigate & analyze digital Evidence using the Autopsy tool.

02. OBJECTIVES:

a) To understand the fundamentals of digital Forensic Investigation

b) To learn how to create & manage a case in Autopsy

c) To acquire & analyze disk images or Storage devices.

d) To recover deleted files & hidden data.

e) To examine artifacts such as web history, files and metadata.

f) To generate a forensic report based on findings.

03. PROCEDURE:

Step 1: Install & launch Autopsy

a. Download & install autopsy from the official website.

b. launch the application.

Step 2: Create a New user

a. click on "New case"

b. Enter:

a. case name

b. Base Directory

c. click Next → Finish.

Step 3: Add Data Source

a. click "Add Data Source"

b. Select one:

• Disk Image (E01, ISO, RAW)

• local Disk

c. Browse & Select the evidence file.

d. click Next.

Step 4: configure Ingest Modules.

a. Enable modules Such as:

- File Type identification
- Keyword Search
- Recent Activity
- Hash lookup

b. click Next → Finish

Step 5: Analyze the Data

a. Wait for ingest process to complete.

b. Navigate through:

- File Views → Browse all files
- Deleted Files → Recover deleted data
- Recent Activity → Browser history, downloads.
- Keyword Hits → Search Suspicious terms.

Step 6: Examine Evidence

a. Open files to inspect:

- Documents.
- Images.
- Videos.

b. check metadata Such as:

- File creation data.
- Last modified data.

Step 7: Recover Files

a. Right-click → Extract File

b. Save to desired location.

Step 8: Generate Report

a. click generate Report

b. Choose Format: HTML / PDF

c. Select required data.

d. Export report.

04. OBSERVATION:

- a. The tool Successfully loaded & processed the data source.
- b. Various Files, including deleted & hidden files were identified.
- c. Browser history, downloads, and user activity were analyzed.
- d. keywords Searches helped identify relevant evidence.
- e. Metadata provided timeline of file activity.
- f. Recovered files were accessible & viewable.

05. RESULT:

The digital Evidence was Successfully analyzed using Autopsy. Important artifacts such as deleted files, browsing history, and file metadata were identified & examined. The investigation demonstrated how Autopsy can be effectively used for forensic analysis & evidence recovery in digital crime investigations.

R.K

PRACTICAL - 08

01. AIM: To analyze & visualize network Traffic data using NetMiner

02. OBJECTIVES:

- a. To understand the basics of network traffic analysis.
- b. To import and preprocess network data in NetMiner.
- c. To visualize network structures such as nodes & edges.
- d. To identify key nodes, clusters, & communication patterns.
- e. To detect abnormal or suspicious traffic behaviour.

03. PROCEDURE:

Step 1: Installation and Setup

- a. Install and launch NetMiner
- b. Create a new project workspace

Step 2: Data collection

- a. Obtain network traffic data
- b. Ensure the dataset contains source destination, and communication details.

Step 3: Import Data

- a. Go to File → Import & load the dataset into NetMiner.
- b. Select the appropriate format

Step 4: Data Preprocessing

- a. Clean the data set (remove duplicates, handle missing values).
- b. Define nodes (devices/IPs) and edges.

Step 5: Network construction

- a. Generate a new graph from the imported data
- b. Assign attributes such as weight or direction.

Step 6: Visualization

- a. Use layout options (eg, circular, force-directed)
- b. Adjust node size, color, & edge thickness based on traffic metrics.

Step 7: Analysis

- a. Perform centrality analysis
- b. Identify clusters or communities in the network.
- c. Detect unusual communication patterns or anomalies.

Step 8: Interpretation

- a. Analyze highly connected nodes and suspicious interactions.
- b. Correlate findings with potential security threats.

Of. OBSERVATIONS:

- a. The network graph displayed multiple nodes representing
- b. Certain nodes showed higher connectivity, indicating heavy traffic activity.
- c. clusters were observed, representing grouped communication patterns.
- d. Some nodes had unusually high traffic, suggesting potential anomalies or suspicious behaviour.
- e. Visualization helped in clearly identifying communication flow & network structure.

05. RESULT:

The network Traffic was successfully analysed using Netminer. The tool enabled effective visualization & interpretation of communication patterns, helping to identify key nodes, clusters, and potential anomalies in the network ~~structure~~ traffic.

Rv